

HOMEWORK 5

Kavya Saranya K V
2025115152

1. Use a linked list for polynomial representation and perform addition and subtraction operations on the polynomials.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int coef, exp;
    struct Node *next;
} Node;

Node* appendTerm(Node *tail, int c, int e) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->coef = c;
    n->exp = e;
    n->next = NULL;
    tail->next = n;
    return n;
}

Node* insertEnd(Node *head, int c, int e) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->coef = c;
    n->exp = e;
    n->next = NULL;
    if (!head)
        return n;
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    return head;
}

Node* combine(Node *a, Node *b, int sign) {
    Node dummy;
    Node *tail = &dummy;
    dummy.next = NULL;
    while (a && b) {
        if (a->exp == b->exp) {
            int c = a->coef + sign * b->coef;
            if (c != 0)
                tail = appendTerm(tail, c, a->exp);
            a = a->next;
        }
    }
}
```

```

        b = b->next;
    } else if (a->exp > b->exp) {
        tail = appendTerm(tail, a->coef, a->exp);
        a = a->next;
    } else {
        tail = appendTerm(tail, sign * b->coef, b->exp);
        b = b->next;
    }
}
while (a) {
    tail = appendTerm(tail, a->coef, a->exp);
    a = a->next;
}
while (b) {
    tail = appendTerm(tail, sign * b->coef, b->exp);
    b = b->next;
}
return dummy.next;
}

Node* add(Node *a, Node *b) { return combine(a, b, 1); }
Node* subtract(Node *a, Node *b) { return combine(a, b, -1); }

void display(char *label, Node *p) {
    printf("%s = ", label);
    if (!p) {
        printf("0\n");
        return;
    }
    int first = 1;
    while (p) {
        if (first) {
            if (p->coef < 0)
                printf("-");
        } else
            printf(" %c ", p->coef < 0 ? '-' : '+');
        if (abs(p->coef) != 1 || p->exp == 0)
            printf("%d", abs(p->coef));
        if (p->exp == 1)
            printf("x");
        else if (p->exp > 1)
            printf("x^%d", p->exp);
        first = 0;
        p = p->next;
    }
    printf("\n");
}

int main() {
    Node *p1 = NULL, *p2 = NULL;
    p1 = insertEnd(p1, 3, 4);
    p1 = insertEnd(p1, 5, 2);
    p1 = insertEnd(p1, 2, 1);
    p1 = insertEnd(p1, 1, 0);

```

```

    p2 = insertEnd(p2, 2, 3);
    p2 = insertEnd(p2, 4, 2);
    p2 = insertEnd(p2, -2, 1);
    p2 = insertEnd(p2, 6, 0);

    display("P1", p1);
    display("P2", p2);
    display("P1 + P2", add(p1, p2));
    display("P1 - P2", subtract(p1, p2));
    display("P1 - P1", subtract(p1, p1));
    return 0;
}

```

Sample Output:

```

P1 = 3x^4 + 5x^2 + 2x + 1
P2 = 2x^3 + 4x^2 - 2x + 6
P1 + P2 = 3x^4 + 2x^3 + 9x^2 + 7
P1 - P2 = 3x^4 - 2x^3 + x^2 + 4x - 5
P1 - P1 = 0

```

2. Find the previous node of a circular linked list.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    if (!head) {
        n->next = n;
        return n;
    }
    Node *last = head;
    while (last->next != head)
        last = last->next;
    last->next = n;
    n->next = head;
    return head;
}

Node* findPrevious(Node *head, int key) {
    if (!head)
        return NULL;
    Node *cur = head;

```

```

    do {
        if (cur->next->data == key)
            return cur;
        cur = cur->next;
    } while (cur != head);
    return NULL;
}

void display(Node *head) {
    Node *cur = head;
    do {
        printf("%d ", cur->data);
        cur = cur->next;
    } while (cur != head);
    printf("\n");
}

int main() {
    Node *head = NULL;
    for (int i = 1; i <= 5; i++)
        head = insertEnd(head, i * 10);
    printf("Circular list: ");
    display(head);

    int keys[] = {30, 10, 50, 99};
    for (int i = 0; i < 4; i++) {
        Node *p = findPrevious(head, keys[i]);
        if (p)
            printf("Previous node of %d = %d\n", keys[i], p->data);
        else
            printf("%d not found in the list\n", keys[i]);
    }
    return 0;
}

```

Sample Output:

```

Circular list: 10 20 30 40 50
Previous node of 30 = 20
Previous node of 10 = 50
Previous node of 50 = 40
99 not found in the list

```

3. Insert at a given position of a circular linked list.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
}

```

```

} Node;

int length(Node *head) {
    if (!head)
        return 0;
    int count = 0;
    Node *cur = head;
    do {
        count++;
        cur = cur->next;
    } while (cur != head);
    return count;
}

Node* insertAt(Node *head, int val, int pos) {
    int len = length(head);
    if (pos < 1 || pos > len + 1) {
        printf("Invalid position %d\n", pos);
        return head;
    }
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    if (!head) {
        n->next = n;
        return n;
    }
    if (pos == 1) {
        Node *last = head;
        while (last->next != head)
            last = last->next;
        n->next = head;
        last->next = n;
        return n;
    }
    Node *p = head;
    for (int i = 1; i < pos - 1; i++)
        p = p->next;
    n->next = p->next;
    p->next = n;
    return head;
}

void display(Node *head) {
    if (!head) {
        printf("(empty)\n");
        return;
    }
    Node *cur = head;
    do {
        printf("%d ", cur->data);
        cur = cur->next;
    } while (cur != head);
    printf("\n");
}

```

```

int main() {
    Node *head = NULL;
    head = insertAt(head, 10, 1);
    head = insertAt(head, 20, 2);
    head = insertAt(head, 30, 3);
    head = insertAt(head, 40, 4);
    printf("Initial list:          ");
    display(head);
    head = insertAt(head, 99, 3);
    printf("Insert 99 at pos 3:    ");
    display(head);
    head = insertAt(head, 55, 1);
    printf("Insert 55 at pos 1:    ");
    display(head);
    head = insertAt(head, 77, 7);
    printf("Insert 77 at pos 7 (end): ");
    display(head);
    head = insertAt(head, 11, 10);
    return 0;
}

```

Sample Output:

```

Initial list:          10 20 30 40
Insert 99 at pos 3:    10 20 99 30 40
Insert 55 at pos 1:    55 10 20 99 30 40
Insert 77 at pos 7 (end): 55 10 20 99 30 40 77
Invalid position 10

```

4. Find the second largest element in a queue.

Code:

```

#include <stdio.h>
#include <limits.h>
#define MAX 10

typedef struct {
    int arr[MAX];
    int front, count;
} Queue;

void enqueue(Queue *q, int val) {
    q->arr[(q->front + q->count) % MAX] = val;
    q->count++;
}

int dequeue(Queue *q) {
    int val = q->arr[q->front];
    q->front = (q->front + 1) % MAX;
    q->count--;
    return val;
}

```

```

}

int secondLargest(Queue *q, int *result) {
    int first = INT_MIN, second = INT_MIN;
    int n = q->count;
    for (int i = 0; i < n; i++) {
        int val = dequeue(q);
        enqueue(q, val);
        if (val > first) {
            second = first;
            first = val;
        } else if (val < first && val > second)
            second = val;
    }
    if (second == INT_MIN)
        return 0;
    *result = second;
    return 1;
}

void display(Queue *q) {
    for (int i = 0; i < q->count; i++)
        printf("%d ", q->arr[(q->front + i) % MAX]);
    printf("\n");
}

int main() {
    Queue q;
    q.front = q.count = 0;
    int vals[] = {12, 35, 1, 10, 34, 35};
    for (int i = 0; i < 6; i++)
        enqueue(&q, vals[i]);
    printf("Queue: ");
    display(&q);
    int res;
    if (secondLargest(&q, &res))
        printf("Second largest element = %d\n", res);
    else
        printf("No second largest element\n");
    printf("Queue after scan: ");
    display(&q);

    Queue q2;
    q2.front = q2.count = 0;
    enqueue(&q2, 7);
    enqueue(&q2, 7);
    printf("\nQueue: ");
    display(&q2);
    if (secondLargest(&q2, &res))
        printf("Second largest element = %d\n", res);
    else
        printf("No second largest element\n");
    return 0;
}

```

Sample Output:

Queue: 12 35 1 10 34 35
Second largest element = 34
Queue after scan: 12 35 1 10 34 35

Queue: 7 7
No second largest element

5. Insert the 3rd element and check whether the 1st and 3rd elements are the same in a linked list.**Code:**

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = NULL;
    if (!head)
        return n;
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    return head;
}

Node* insertAt(Node *head, int val, int pos) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    if (pos == 1) {
        n->next = head;
        return n;
    }
    Node *p = head;
    for (int i = 1; i < pos - 1 && p; i++)
        p = p->next;
    if (!p) {
        printf("Invalid position\n");
        free(n);
        return head;
    }
    n->next = p->next;
    p->next = n;
}
```

```

        return head;
    }

void display(Node *head) {
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int firstEqualsThird(Node *head) {
    if (!head || !head->next || !head->next->next)
        return 0;
    return head->data == head->next->next->data;
}

int main() {
    int arr[] = {10, 20, 30, 40};

    Node *a = NULL;
    for (int i = 0; i < 4; i++)
        a = insertEnd(a, arr[i]);
    printf("List: ");
    display(a);
    a = insertAt(a, 10, 3);
    printf("After inserting 10 as 3rd element: ");
    display(a);
    printf("1st and 3rd elements are %s\n\n", firstEqualsThird(a) ? "same" :
"not same");

    Node *b = NULL;
    for (int i = 0; i < 4; i++)
        b = insertEnd(b, arr[i]);
    printf("List: ");
    display(b);
    b = insertAt(b, 99, 3);
    printf("After inserting 99 as 3rd element: ");
    display(b);
    printf("1st and 3rd elements are %s\n", firstEqualsThird(b) ? "same" : "not
same");
    return 0;
}

```

Sample Output:

```

List: 10 20 30 40
After inserting 10 as 3rd element: 10 20 10 30 40
1st and 3rd elements are same

List: 10 20 30 40
After inserting 99 as 3rd element: 10 20 99 30 40
1st and 3rd elements are not same

```

6. Implement two stacks in one array. Write routines to traverse the array and display.

Code:

```
#include <stdio.h>
#define MAX 8

typedef struct {
    int arr[MAX];
    int top1, top2;
} TwoStacks;

void init(TwoStacks *s) {
    s->top1 = -1;
    s->top2 = MAX;
}

void push1(TwoStacks *s, int val) {
    if (s->top1 + 1 == s->top2) {
        printf("Overflow: cannot push %d to stack 1\n", val);
        return;
    }
    s->arr[++(s->top1)] = val;
}

void push2(TwoStacks *s, int val) {
    if (s->top1 + 1 == s->top2) {
        printf("Overflow: cannot push %d to stack 2\n", val);
        return;
    }
    s->arr[--(s->top2)] = val;
}

int pop1(TwoStacks *s) {
    if (s->top1 == -1) {
        printf("Stack 1 underflow\n");
        return -1;
    }
    return s->arr[(s->top1)--];
}

int pop2(TwoStacks *s) {
    if (s->top2 == MAX) {
        printf("Stack 2 underflow\n");
        return -1;
    }
    return s->arr[(s->top2)++];
}

void display(TwoStacks *s) {
    printf("Array   : ");
    for (int i = 0; i < MAX; i++) {
        if (i <= s->top1 || i >= s->top2)
            printf("%d ", s->arr[i]);
    }
}
```

```

        else
            printf("_ ");
    }
    printf("\nStack 1 (top to bottom): ");
    for (int i = s->top1; i >= 0; i--)
        printf("%d ", s->arr[i]);
    printf("\nStack 2 (top to bottom): ");
    for (int i = s->top2; i < MAX; i++)
        printf("%d ", s->arr[i]);
    printf("\n");
}

int main() {
    TwoStacks s;
    init(&s);
    push1(&s, 10);
    push1(&s, 20);
    push1(&s, 30);
    push1(&s, 40);
    push2(&s, 100);
    push2(&s, 200);
    push2(&s, 300);
    display(&s);

    printf("\nPopped from stack 1: %d\n", pop1(&s));
    printf("Popped from stack 2: %d\n", pop2(&s));
    display(&s);

    printf("\n");
    push1(&s, 41);
    push1(&s, 42);
    push1(&s, 43);
    push2(&s, 500);
    display(&s);
    return 0;
}

```

Sample Output:

```

Array   : 10 20 30 40 _ 300 200 100
Stack 1 (top to bottom): 40 30 20 10
Stack 2 (top to bottom): 300 200 100

```

```

Popped from stack 1: 40
Popped from stack 2: 300
Array   : 10 20 30 _ _ 200 100
Stack 1 (top to bottom): 30 20 10
Stack 2 (top to bottom): 200 100

```

```

Overflow: cannot push 500 to stack 2
Array   : 10 20 30 41 42 43 200 100
Stack 1 (top to bottom): 43 42 41 30 20 10
Stack 2 (top to bottom): 200 100

```

7. Given a linked list containing n head nodes where every node in the linked list contains two pointers: (i) next points to the next node in the list, (ii) bottom points to a sub linked list where the current node is the head. Each of the sub-linked list nodes and the head nodes are sorted in ascending order based on their data. Flatten the linked list so that all the nodes appear in a single linked list while maintaining the sorted order.

Input: 5 -> 10 -> 19 -> 28 (bottom lists: 5: 7, 8; 10: 20; 19: 22; 28: 40, 45)

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next, *bottom;
} Node;

Node* newNode(int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = n->bottom = NULL;
    return n;
}

Node* merge(Node *a, Node *b) {
    if (!a)
        return b;
    if (!b)
        return a;
    Node *result;
    if (a->data < b->data) {
        result = a;
        result->bottom = merge(a->bottom, b);
    } else {
        result = b;
        result->bottom = merge(a, b->bottom);
    }
    result->next = NULL;
    return result;
}

Node* flatten(Node *head) {
    if (!head || !head->next)
        return head;
    head->next = flatten(head->next);
    return merge(head, head->next);
}

int main() {
    Node *head = newNode(5);
    head->bottom = newNode(7);
    head->bottom->bottom = newNode(8);
```

```

head->next = newNode(10);
head->next->bottom = newNode(20);

head->next->next = newNode(19);
head->next->next->bottom = newNode(22);

head->next->next->next = newNode(28);
head->next->next->next->bottom = newNode(40);
head->next->next->next->bottom->bottom = newNode(45);

head = flatten(head);

printf("Flattened list: ");
for (Node *p = head; p; p = p->bottom) {
    printf("%d", p->data);
    if (p->bottom)
        printf(" -> ");
}
printf("\n");
return 0;
}

```

Sample Output:

Flattened list: 5 -> 7 -> 8 -> 10 -> 19 -> 20 -> 22 -> 28 -> 40 -> 45

8. Given k sorted linked lists of different sizes, merge them into a single list maintaining the sorted order.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = NULL;
    if (!head)
        return n;
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    return head;
}

Node* mergeTwo(Node *a, Node *b) {

```

```

Node dummy;
Node *tail = &dummy;
dummy.next = NULL;
while (a && b) {
    if (a->data <= b->data) {
        tail->next = a;
        a = a->next;
    } else {
        tail->next = b;
        b = b->next;
    }
    tail = tail->next;
}
tail->next = a ? a : b;
return dummy.next;
}

Node* mergeK(Node *lists[], int k) {
    if (k == 0)
        return NULL;
    for (int step = 1; step < k; step *= 2)
        for (int i = 0; i + step < k; i += 2 * step)
            lists[i] = mergeTwo(lists[i], lists[i + step]);
    return lists[0];
}

void display(Node *head) {
    while (head) {
        printf("%d", head->data);
        if (head->next)
            printf(" -> ");
        head = head->next;
    }
    printf("\n");
}

int main() {
    int a[] = {1, 4, 7};
    int b[] = {2, 5};
    int c[] = {3, 6, 8, 9};
    int d[] = {0, 10};
    Node *lists[4] = {NULL, NULL, NULL, NULL};
    for (int i = 0; i < 3; i++)
        lists[0] = insertEnd(lists[0], a[i]);
    for (int i = 0; i < 2; i++)
        lists[1] = insertEnd(lists[1], b[i]);
    for (int i = 0; i < 4; i++)
        lists[2] = insertEnd(lists[2], c[i]);
    for (int i = 0; i < 2; i++)
        lists[3] = insertEnd(lists[3], d[i]);

    for (int i = 0; i < 4; i++) {
        printf("List %d: ", i + 1);
        display(lists[i]);
    }
}

```

```

    }
    Node *merged = mergeK(lists, 4);
    printf("Merged : ");
    display(merged);
    return 0;
}

```

Sample Output:

```

List 1: 1 -> 4 -> 7
List 2: 2 -> 5
List 3: 3 -> 6 -> 8 -> 9
List 4: 0 -> 10
Merged : 0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10

```

9. Swap the kth node from the beginning with the kth node from the end in a linked list.

Code:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node* insertEnd(Node *head, int val) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->data = val;
    n->next = NULL;
    if (!head)
        return n;
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    return head;
}

int length(Node *head) {
    int n = 0;
    while (head) {
        n++;
        head = head->next;
    }
    return n;
}

Node* swapKth(Node *head, int k) {
    int n = length(head);
    if (k < 1 || k > n || 2 * k - 1 == n)
        return head;
}

```

```

int x = k, y = n - k + 1;
if (x > y) {
    int t = x;
    x = y;
    y = t;
}
Node *prevX = NULL, *curX = head;
for (int i = 1; i < x; i++) {
    prevX = curX;
    curX = curX->next;
}
Node *prevY = NULL, *curY = head;
for (int i = 1; i < y; i++) {
    prevY = curY;
    curY = curY->next;
}

if (curX->next == curY) {
    curX->next = curY->next;
    curY->next = curX;
    if (prevX)
        prevX->next = curY;
    else
        head = curY;
    return head;
}
if (prevX)
    prevX->next = curY;
else
    head = curY;
prevY->next = curX;
Node *t = curX->next;
curX->next = curY->next;
curY->next = t;
return head;
}

void display(Node *head) {
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

Node* build(int n) {
    Node *h = NULL;
    for (int i = 1; i <= n; i++)
        h = insertEnd(h, i);
    return h;
}

int main() {
    Node *h = build(8);

```

```

    printf("List:      ");
    display(h);
    h = swapKth(h, 2);
    printf("k = 2:      ");
    display(h);

    h = build(8);
    h = swapKth(h, 1);
    printf("k = 1:      ");
    display(h);

    h = build(8);
    h = swapKth(h, 4);
    printf("k = 4 (adjacent nodes): ");
    display(h);

    h = build(7);
    h = swapKth(h, 4);
    printf("k = 4 on 7 nodes (same node, no change): ");
    display(h);

    h = build(5);
    h = swapKth(h, 9);
    printf("k = 9 on 5 nodes (k too large, no change): ");
    display(h);
    return 0;
}

```

Sample Output:

```

List:      1 2 3 4 5 6 7 8
k = 2:      1 7 3 4 5 6 2 8
k = 1:      8 2 3 4 5 6 7 1
k = 4 (adjacent nodes): 1 2 3 5 4 6 7 8
k = 4 on 7 nodes (same node, no change): 1 2 3 4 5 6 7
k = 9 on 5 nodes (k too large, no change): 1 2 3 4 5

```

10. Multiply two numbers represented by two linked lists. Example: 1 -> 0 -> 0 x 1 -> 0 = 1 -> 0 -> 0 -> 0.

Code:

```

#include <stdio.h>
#include <stdlib.h>
#define MAX 100

typedef struct Node {
    int digit;
    struct Node *next;
} Node;

Node* insertEnd(Node *head, int d) {
    Node *n = (Node*)malloc(sizeof(Node));

```

```

    n->digit = d;
    n->next = NULL;
    if (!head)
        return n;
    Node *t = head;
    while (t->next)
        t = t->next;
    t->next = n;
    return head;
}

int toArray(Node *head, int arr[]) {
    int n = 0;
    while (head) {
        arr[n++] = head->digit;
        head = head->next;
    }
    return n;
}

Node* multiply(Node *l1, Node *l2) {
    int a[MAX], b[MAX], res[2 * MAX] = {0};
    int n = toArray(l1, a), m = toArray(l2, b);

    for (int i = n - 1; i >= 0; i--)
        for (int j = m - 1; j >= 0; j--)
            res[i + j + 1] += a[i] * b[j];

    for (int i = n + m - 1; i > 0; i--) {
        res[i - 1] += res[i] / 10;
        res[i] %= 10;
    }

    int start = 0;
    while (start < n + m - 1 && res[start] == 0)
        start++;

    Node *result = NULL;
    for (int i = start; i < n + m; i++)
        result = insertEnd(result, res[i]);
    return result;
}

void display(Node *head) {
    while (head) {
        printf("%d", head->digit);
        if (head->next)
            printf(" -> ");
        head = head->next;
    }
}

Node* build(int arr[], int n) {
    Node *h = NULL;

```

```

        for (int i = 0; i < n; i++)
            h = insertEnd(h, arr[i]);
        return h;
    }

int main() {
    int x1[] = {1, 0, 0}, y1[] = {1, 0};
    Node *a = build(x1, 3), *b = build(y1, 2);
    display(a);
    printf("  x  ");
    display(b);
    printf("  =  ");
    display(multiply(a, b));
    printf("\n");

    int x2[] = {9, 9, 9}, y2[] = {9, 9};
    a = build(x2, 3);
    b = build(y2, 2);
    display(a);
    printf("  x  ");
    display(b);
    printf("  =  ");
    display(multiply(a, b));
    printf("\n");

    int x3[] = {0}, y3[] = {4, 5};
    a = build(x3, 1);
    b = build(y3, 2);
    display(a);
    printf("  x  ");
    display(b);
    printf("  =  ");
    display(multiply(a, b));
    printf("\n");
    return 0;
}

```

Sample Output:

```

1 -> 0 -> 0  x  1 -> 0  =  1 -> 0 -> 0 -> 0
9 -> 9 -> 9  x  9 -> 9  =  9 -> 8 -> 9 -> 0 -> 1
0  x  4 -> 5  =  0

```